

Variable-Length Message-Passing Graph Neural Network as a Surrogate for a 1D Finite Element Bead Chain Simulation

Grey L. Goodwin · Daniel L. Lau, PhD.

Abstract

Finite element simulations of flexible structures require thousands of per-timestep computations. Graph neural networks (GNNs) are a natural fit because the discretized mesh is inherently a graph: beads are nodes, rods are edges. An initial shared MLP trained on a fixed 16-bead chain achieved accurate single-step predictions but could not generalize to chains of other lengths. We restructure the surrogate as a true message-passing GNN with separate node/edge encoders, learned messages per rod, and ghost masking for endpoints. Because all weights are shared, one trained model handles chains of any length. We train on one million volume-sampled scenarios from chains of 8–32 beads and evaluate on chain lengths, orientations, and velocities never seen.

Problem Setup

A whip is modeled as a 1D chain of N tapered mass nodes connected by $N - 1$ spring rods, anchored at one end. Each free bead experiences gravity, Hookean spring forces, rod-axis damping, and velocity drag. Time integration uses symplectic Euler:

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + \Delta t \mathbf{a}(\mathbf{x}^n) \quad (1)$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + \Delta t \mathbf{v}_i^{n+1} \quad (2)$$

The force on bead i depends only on its state and those of its immediate neighbors—a locality that motivates a 1-hop graph architecture.

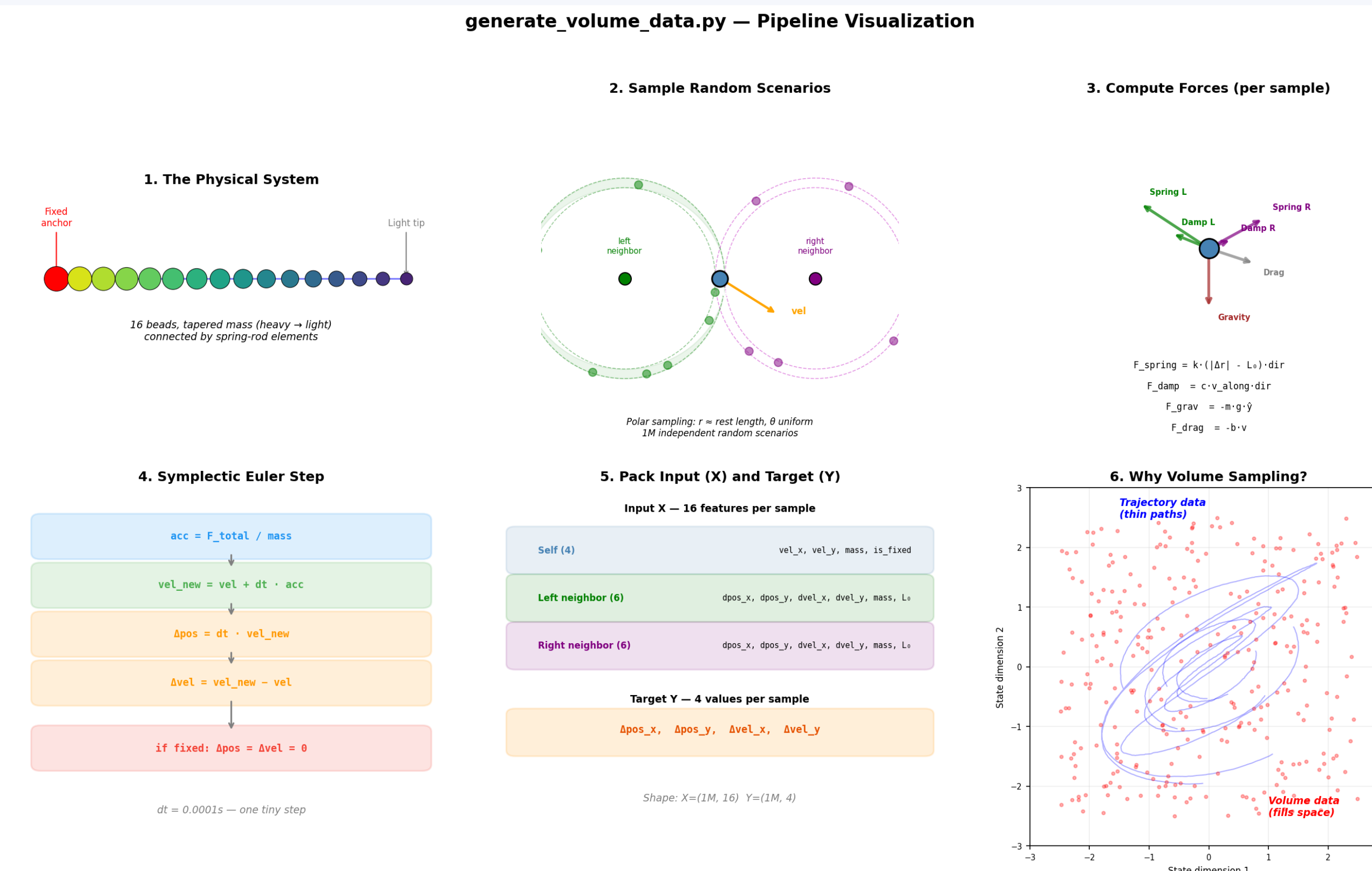
MPGNN Architecture

- **Node encoder** ($4 \rightarrow 64$): $[v_x, v_y, m, \text{fixed}]$
- **Edge encoder** ($6 \rightarrow 64$): shared left/right, relative features
- **Message MLP** ($64 \rightarrow 64$): per-edge learned message
- **Ghost mask**: zero messages from absent neighbors
- **Node update MLP** ($192 \rightarrow 64$): fuses self + messages
- **Output head** ($64 \rightarrow 4$): residual state deltas

80,324 parameters, shared across all beads, independent of chain length.

Data Pipeline: Volume-Sampled Per-Bead Physics

Rather than recording trajectories, we sample random single-step scenarios uniformly across the reachable state space. Each sample is one bead at one instant plus the analytical answer one timestep later.

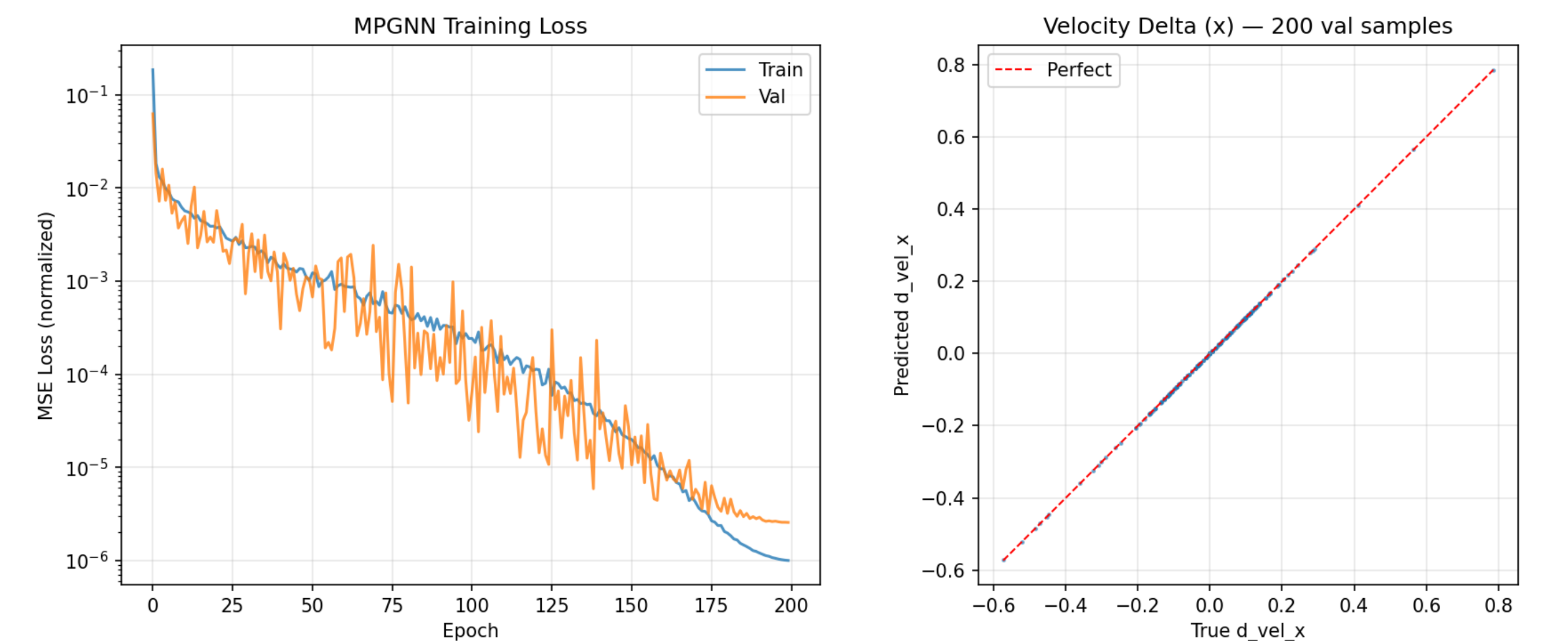


- (1) Tapered chain with rods. (2) Neighbor positions drawn in polar coordinates. (3) Forces computed on central bead. (4) Symplectic Euler step. (5) Inputs (16 features) and targets (4 deltas) packed. (6) Volume sampling covers state space uniformly.

Why Volume Sampling?

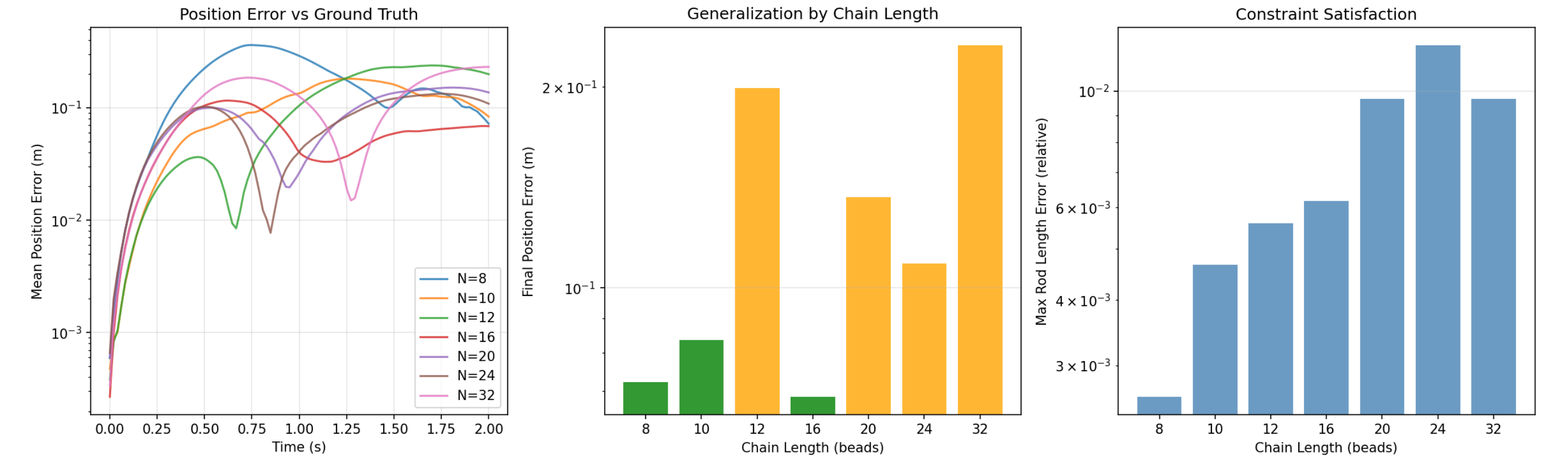
Trajectory data traces thin curves through state space; small rollout errors push the simulation off those curves into regions without training coverage. Volume sampling broadens the training distribution so the model sees a dense cross-section of every plausible state. Combined with randomized chain lengths ($N \in [8, 32]$) and taper ratios ($r \in [1, 10]$), this decouples the training data from any specific chain configuration.

Training Convergence



Train/validation MSE loss over 200 epochs (left) and predicted vs. true Δv_x on validation samples (right). Predictions fall tightly along the diagonal, indicating accurate single-step physics.

Generalization Across Chain Lengths



Trained MPGNN evaluated on chains of $N \in \{8, 10, 12, 16, 20, 24, 32\}$ via full rollouts. Short and medium chains track closely; longer chains accumulate more drift.

Conclusion and Future Work

Swapping a fixed-size MLP for a weight-shared message-passing GNN enables one trained model to simulate bead chains of any length. Separate node/edge encoders, per-rod messages, and ghost masking give a size-agnostic, physically structured surrogate. Volume sampling provides the coverage needed for rollout generalization.

Future work: rollout training to reduce drift, constraint loss or hybrid SHAKE correction, and deeper message-passing on multi-hop data.